

Lab3 Report

Kafka + Spark Structured Streaming ETL Pipeline

Step 0: Environment Setup

0a Start the Kafka Cluster

Initializes the local Kafka environment containing the brokers required for the source data.

```
docker compose up -d
docker compose ps
docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --list
```

```
Container kafka3 Starting 0.9s
Volume lab3_kafka1_data Created 0.0s
Container kafka1 Starting 0.9s
Running 7/8 kafka2_data Created 0.0s
Network lab3_default Created 0.0s
Container kafka3 Started 1.0s
Volume lab3_kafka3_data Created 0.0s
Container kafka2 Started 0.9s
Volume lab3_kafka1_data Created 0.0s
Container kafka1 Started 0.9s
Running 7/8 kafka2_data Created 0.0s
Network lab3_default Created 0.0s
Container kafka3 Started 1.0s
Volume lab3_kafka3_data Created 0.0s
Container kafka2 Started 0.9s
Volume lab3_kafka1_data Created 0.0s
Container kafka1 Started 0.9s
Running 8/8 kafka2_data Created 0.0s
Network lab3_default Created 0.0s
Container kafka3 Started 1.0s
Volume lab3_kafka3_data Created 0.0s
Container kafka2 Started 0.9s
Volume lab3_kafka1_data Created 0.0s
Container kafka1 Started 0.9s
Volume lab3_kafka2_data Created 0.0s
Container kafka-ui Starting 0.9s
Container kafka3 Started 1.0s
Container kafka2 Started 0.9s
Container kafka1 Started 0.9s
Container kafka-ui Started 1.0s
(bigdata) PS D:\VEFREI\Data_Engineering\LAB3> docker compose ps
time="2026-05-13T09:50:32+08:00" level=warning msg="D:\VEFREI\Data_Engineering\LAB3\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to a
void potential confusion"
NAME        IMAGE                                COMMAND                                SERVICE    CREATED      STATUS      PORTS
kafka-ui    provectuslabs/kafka-ui:latest      "/bin/sh -c 'java -D..."          kafka-ui    10 seconds ago Up 9 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
kafka1      confluentinc/cp-kafka:7.5.0        "/etc/confluent/dock..."          kafka1     10 seconds ago Up 9 seconds 0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
kafka2      confluentinc/cp-kafka:7.5.0        "/etc/confluent/dock..."          kafka2     10 seconds ago Up 9 seconds 0.0.0.0:9094->9094/tcp, [::]:9094->9094/tcp
kafka3      confluentinc/cp-kafka:7.5.0        "/etc/confluent/dock..."          kafka3     10 seconds ago Up 9 seconds 0.0.0.0:9096->9096/tcp, [::]:9096->9096/tcp
(bigdata) PS D:\VEFREI\Data_Engineering\LAB3> docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --list
(bigdata) PS D:\VEFREI\Data_Engineering\LAB3>
```

0b Install Python Dependencies

This prepares the Python virtual environment with the necessary libraries (PySpark and Kafka Python client) to run the ETL pipeline. I'm already in the virtual environment, so I don't need to activate it.

```
pip install pyspark==3.5.3 kafka-python-ng
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB3> pip install pyspark==3.5.3 kafka-python-ng
Collecting pyspark==3.5.3
  Downloading pyspark-3.5.3.tar.gz (317.3 MB)
    35.7/317.3 MB 75.3 kB/s 1:02:19
  WARNING: Attempting to resume incomplete download (35.7 MB/317.3 MB, attempt 1)
  Resuming download pyspark-3.5.3.tar.gz (35.7 MB/317.3 MB)
    317.3/317.3 MB 1.2 MB/s 0:04:25
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: kafka-python-ng in d:\app\anaconda\envs\bigdata\lib\site-packages (2.2.3)
Collecting py4j==0.10.9.7 (from pyspark==3.5.3)
  Downloading py4j-0.10.9.7-py2.py3-none-any.whl.metadata (1.5 kB)
  Downloading py4j-0.10.9.7-py2.py3-none-any.whl (200 kB)
Building wheels for collected packages: pyspark
  Building wheel for pyspark (pyproject.toml) ... done
  Created wheel for pyspark: filename=pyspark-3.5.3-py2.py3-none-any.whl size=317840734 sha256=82f3fabfd607acfa8223f619ee993a9fb2e87bb86bb2fc110dda6a2e9a0b1506
  Stored in directory: c:\users\xback20040219\appdata\local\pip\cache\wheels\2e\d2\18\6f4f20e8332359f7fffceb6828edcc80ef96f86744192a7bb9
Successfully built pyspark
Installing collected packages: py4j, pyspark
  Attempting uninstall: py4j
    Found existing installation: py4j 0.10.9.9
    Uninstalling py4j-0.10.9.9:
      Successfully uninstalled py4j-0.10.9.9
  Attempting uninstall: pyspark
    Found existing installation: pyspark 4.0.2
    Uninstalling pyspark-4.0.2:
      Successfully uninstalled pyspark-4.0.2
Successfully installed py4j-0.10.9.7 pyspark-3.5.3
(bigdata) PS D:\EFREI\Data_Engineering\LAB3>
```

0c Create Output Directory

Creates the local file system directories where the pipeline will store its final Parquet output files and its operational streaming state (checkpoint).

```
mkdir spark-etl\output
mkdir spark-etl\checkpoint
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB3> mkdir spark-etl\output
```

```
目录: D:\EFREI\Data_Engineering\LAB3\spark-etl
```

Mode	LastWriteTime	Length	Name
d-----	2026/5/13 10:43		output

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB3> mkdir spark-etl\checkpoint
```

```
目录: D:\EFREI\Data_Engineering\LAB3\spark-etl
```

Mode	LastWriteTime	Length	Name
d-----	2026/5/13 10:44		checkpoint

Establishing a dedicated checkpoint directory is crucial; without it, a restarted Spark job cannot resume properly and would reprocess data from the beginning.

Step 1: Create the Spark Session

1a Imports

Create a file `etl_pipeline.py` in vscode.

Imports all required classes, functions, and data types from the PySpark library to build the pipeline.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, avg, window, to_timestamp, expr, lit
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, LongType
```

```
etl_pipeline.py
1  # 1a. Import necessary libraries
2  from pyspark.sql import SparkSession
3  from pyspark.sql.functions import col, from_json, avg, window, to_timestamp, expr, lit
4  from pyspark.sql.types import StructType, StructField, StringType, DoubleType, LongType
```

1b Build the SparkSession

Initializes the Spark application, configures it to run locally, and tells it to download the necessary Kafka connector JARs.

```
spark = SparkSession.builder \
    .appName("SessionETLPipeline") \
    .master("local[*]") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3") \
    .config("spark.sql.shuffle.partitions", "3") \
    .config("spark.streaming.stopGracefullyOnShutdown", "true") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")
```

```
6  # 1b. Create a Spark session
7  spark = SparkSession.builder \
8      .appName("SessionETLPipeline") \
9      .master("local[*]") \
10     .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3") \
11     .config("spark.sql.shuffle.partitions", "3") \
12     .config("spark.streaming.stopGracefullyOnShutdown", "true") \
13     .getOrCreate()
14
15  spark.sparkContext.setLogLevel("WARN")
16
```

Step 2: Read the Raw Kafka Stream

Read the Raw Kafka Stream

Connects Spark to the Kafka cluster to continuously read events from the `sensor-events` topic starting from the earliest offset.

```
KAFKA_BROKERS = "localhost:9092,localhost:9094,localhost:9096"
TOPIC = "sensor-events"

raw_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", KAFKA_BROKERS) \
    .option("subscribe", TOPIC) \
    .option("startingOffsets", "earliest") \
    .option("failOnDataLoss", "false") \
    .load()

raw_stream.printSchema()
```

```
17  # 2. Read the Raw Kafka Stream
18  KAFKA_BROKERS = "localhost:9092,localhost:9094,localhost:9096"
19  TOPIC = "sensor-events"
20
21  raw_stream = spark.readStream \
22      .format("kafka") \
23      .option("kafka.bootstrap.servers", KAFKA_BROKERS) \
24      .option("subscribe", TOPIC) \
25      .option("startingOffsets", "earliest") \
26      .option("failOnDataLoss", "false") \
27      .load()
28
29  raw_stream.printSchema()
```

Step 3: Parse the JSON Payload

3a Define the Expected Schema

Explicitly defines the structure and data types of the JSON payload arriving from Kafka.

```
sensor_schema = StructType([
    StructField("sensor", StringType(), nullable=False),
    StructField("value", DoubleType(), nullable=False),
    StructField("unit", StringType(), nullable=True),
    StructField("timestamp", LongType(), nullable=False),
    StructField("source", StringType(), nullable=True),
])
```

```

31 # 3a. Define the Expected Schema
32 sensor_schema = StructType([
33     StructField("sensor", StringType(), nullable=False),
34     StructField("value", DoubleType(), nullable=False),
35     StructField("unit", StringType(), nullable=True),
36     StructField("timestamp", LongType(), nullable=False),
37     StructField("source", StringType(), nullable=True),
38 ])

```

3b Cast and Parse

Casts the raw binary Kafka payload to a string, parses it into a structured format using the schema, and extracts the time into a Spark-compatible timestamp.

```

parsed_stream = raw_stream \
    .select(
        col("key").cast("string").alias("message_key"),
        from_json(col("value").cast("string"), sensor_schema).alias("data"),
        col("partition"),
        col("offset"),
        col("timestamp").alias("kafka_timestamp")
    ) \
    .select(
        col("message_key"),
        col("data.sensor"),
        col("data.value"),
        col("data.unit"),
        to_timestamp(expr("data.timestamp / 1000")).alias("event_time"),
        col("partition"),
        col("offset")
    )

```

```

40 # 3b. Cast and Parse
41 parsed_stream = raw_stream \
42     .select(
43         col("key").cast("string").alias("message_key"),
44         from_json(col("value").cast("string"), sensor_schema).alias("data"),
45         col("partition"),
46         col("offset"),
47         col("timestamp").alias("kafka_timestamp")
48     ) \
49     .select(
50         col("message_key"),
51         col("data.sensor"),
52         col("data.value"),
53         col("data.unit"),
54         to_timestamp(expr("data.timestamp / 1000")).alias("event_time"),
55         col("partition"),
56         col("offset")
57     )

```

Step 4: Apply Business Logic: Filter and Alert

Apply Business Logic

Cleans the data by dropping bad records (nulls) and flags sensor readings that exceed predefined safety thresholds.

```
# 4a: Filter out null records
clean_stream = parsed_stream \
    .filter(col("sensor").isNotNull()) \
    .filter(col("value").isNotNull())

# 4b: Flag anomalies
TEMP_THRESHOLD = 35.0
HUM_THRESHOLD = 90.0

flagged_stream = clean_stream.withColumn(
    "is_anomaly",
    (
        ((col("sensor") == "temperature") & (col("value") > TEMP_THRESHOLD)) |
        ((col("sensor") == "humidity") & (col("value") > HUM_THRESHOLD))
    )
)
```

`withColumn` adds a boolean column without materializing anything immediately; it adds a node to the lazy evaluation DAG.

```
59 # 4a. Filter out null records
60 clean_stream = parsed_stream \
61     .filter(col("sensor").isNotNull()) \
62     .filter(col("value").isNotNull())
63
64 # 4b. Flag anomalies
65 TEMP_THRESHOLD = 35.0
66 HUM_THRESHOLD = 90.0
67
68 flagged_stream = clean_stream.withColumn(
69     "is_anomaly",
70     (
71         ((col("sensor") == "temperature") & (col("value") > TEMP_THRESHOLD)) |
72         ((col("sensor") == "humidity") & (col("value") > HUM_THRESHOLD))
73     )
74 )
```

Step 5: Windowed Aggregation

Windowed Aggregation

Groups the incoming stream into 5-minute time buckets based on the actual event time, calculates the average value, and applies a watermark to handle late-arriving data.

```
# 5a: Add watermark
watermarked = flagged_stream \
    .withWatermark("event_time", "2 minutes")

# 5b: Group by 5-minute window
windowed_avg = watermarked \
    .groupBy(
        window(col("event_time"), "5 minutes"),
        col("sensor")
    ) \
    .agg(
        avg(col("value")).alias("avg_value")
    )
```

The watermark of "2 minutes" drops events arriving more than 2 minutes behind the latest seen event time, allowing Spark to close windows and free memory.

```
76 # 5a. Add watermark
77 watermarked = flagged_stream \
78 |     .withWatermark("event_time", "2 minutes")
79
80 # 5b. Group by 5-minute window
81 windowed_avg = watermarked \
82 |     .groupBy(
83 |         window(col("event_time"), "5 minutes"),
84 |         col("sensor")
85 |     ) \
86 |     .agg(
87 |         avg(col("value")).alias("avg_value")
88 |     )
```

Step 6: Write to Parquet Sink

6a Write the Aggregated Stream

Starts the streaming query, continuously writing the updated 5-minute averages into a Parquet directory every 10 seconds.

```
OUTPUT_PATH = "./spark-etl/output"
CHECKPOINT_PATH = "./spark-etl/checkpoint"

query = windowed_avg \
    .writeStream \
    .outputMode("append") \
    .format("parquet") \
    .option("path", OUTPUT_PATH) \
    .option("checkpointLocation", CHECKPOINT_PATH) \
    .trigger(processingTime="10 seconds") \
    .start()
```

`outputMode("update")` writes only rows whose aggregated value changed.

The checkpoint tracks offsets and state to prevent reprocessing upon restart .

```
93  # 6a. Write the Aggregated Stream
94  OUTPUT_PATH = "./spark-etl/output"
95  CHECKPOINT_PATH = "./spark-etl/checkpoint"
96
97  query = windowed_avg \
98      .writeStream \
99      .outputMode("append") \
100     .format("parquet") \
101     .option("path", OUTPUT_PATH) \
102     .option("checkpointLocation", CHECKPOINT_PATH) \
103     .trigger(processingTime="10 seconds") \
104     .start()
```

6b Write the Raw Flagged Stream (Optional)

A dual-write pattern demonstrating how multiple queries can run concurrently on the same SparkSession, writing the raw, non-aggregated records to a separate directory.

```
RAW_OUTPUT = "./spark-etl/raw"
RAW_CHECKPOINT = "./spark-etl/checkpoint-raw"

query_raw = flagged_stream \
    .writeStream \
    .outputMode("append") \
    .format("parquet") \
```



```
.option("path", RAW_OUTPUT) \  
.option("checkpointLocation", RAW_CHECKPOINT) \  
.trigger(processingTime="10 seconds") \  
.start()
```

```
print("Streaming query started. Press Ctrl+C to stop.")  
spark.streams.awaitAnyTermination()
```

```
100  
107 # 6b. Write the Raw Flagged Stream  
108 RAW_OUTPUT = "./spark-etl/raw"  
109 RAW_CHECKPOINT = "./spark-etl/checkpoint-raw"  
110  
111 query_raw = flagged_stream \  
112     .writeStream \  
113     .outputMode("append") \  
114     .format("parquet") \  
115     .option("path", RAW_OUTPUT) \  
116     .option("checkpointLocation", RAW_CHECKPOINT) \  
117     .trigger(processingTime="10 seconds") \  
118     .start()  
119  
120 print("Streaming query started. Press Ctrl+C to stop.")  
121 spark.streams.awaitAnyTermination()
```

Step 7: Run the Full Pipeline

7a Start the Spark Job

Submits the completed Python script (`etl_pipeline.py`) to the Spark engine via the command line.

When you run `python etl_pipeline.py`, Spark will start in the background and automatically handle the package dependencies, so there is no need to submit it.

```
$env:JAVA_HOME=""  
python etl_pipeline.py
```

```

org.apache.spark#spark-token-provider-kafka-0-10_2.12:0.9.0 from central in [default]
org.lz4#lz4-java:1.8.0 from central in [default]
org.slf4j#slf4j-api:2.0.7 from central in [default]
org.xerial.snappy#snappy-java:1.1.10.5 from central in [default]

```

conf	number	modules			artifacts	
		search	downloaded	evicted	number	downloaded
default	11	0	0	0	11	0

```

:: retrieving :: org.apache.spark#spark-submit-parent-0fbdb298-cbc7-4088-a09f-8663cf0c3e38
  confs: [default]
  0 artifacts copied, 11 already retrieved (0kB/53ms)
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
root
-- key: binary (nullable = true)
-- value: binary (nullable = true)
-- topic: string (nullable = true)
-- partition: integer (nullable = true)
-- offset: long (nullable = true)
-- timestamp: timestamp (nullable = true)
-- timestampType: integer (nullable = true)
26/05/13 17:06:57 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
26/05/13 17:06:58 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
Streaming query started. Press Ctrl+C to stop.
26/05/13 17:06:58 WARN AdminClientConfig: These configurations '[key.deserializer, value.deserializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.
26/05/13 17:06:58 WARN AdminClientConfig: These configurations '[key.deserializer, value.deserializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.
26/05/13 17:08:24 WARN HDFSBackedStateStoreProvider: The state for version 4 doesn't exist in loadedMaps. Reading snapshot file and delta files if needed...Note that this is normal for the first batch of starting query.
26/05/13 17:08:24 WARN HDFSBackedStateStoreProvider: The state for version 4 doesn't exist in loadedMaps. Reading snapshot file and delta files if needed...Note that this is normal for the first batch of starting query.
26/05/13 17:08:24 WARN HDFSBackedStateStoreProvider: The state for version 4 doesn't exist in loadedMaps. Reading snapshot file and delta files if needed...Note that this is normal for the first batch of starting query.

```

7b Send Messages from the Producer

Uses a separate terminal and script to publish mock sensor data into the Kafka topic.

```
python producer.py
```

```
(bigdata) PS C:\Users\xback20040219> cd "D:\EFREI\Data_Engineering\LAB3"
(bigdata) PS D:\EFREI\Data_Engineering\LAB3> python producer.py
Sent to partition 0, offset 12
Sent to partition 2, offset 38
Sent to partition 0, offset 13
Sent to partition 2, offset 39
Sent to partition 2, offset 40
Sent to partition 0, offset 14
Sent to partition 2, offset 41
Sent to partition 0, offset 15
Sent to partition 2, offset 42
Sent to partition 2, offset 43
Sent to partition 2, offset 44
Sent to partition 0, offset 16
Sent to partition 0, offset 17
Sent to partition 2, offset 45
Sent to partition 0, offset 18
Sent to partition 0, offset 19
Sent to partition 2, offset 46
Sent to partition 2, offset 47
Sent to partition 0, offset 20
Sent to partition 2, offset 48
Sent to partition 0, offset 21
Sent to partition 2, offset 49
Sent to partition 0, offset 22
Sent to partition 2, offset 50
Sent to partition 0, offset 23
Sent to partition 2, offset 51
Sent to partition 2, offset 52
Sent to partition 0, offset 24
Sent to partition 2, offset 53
Sent to partition 2, offset 54
Sent to partition 2, offset 55
Sent to partition 2, offset 56
Sent to partition 2, offset 57
Sent to partition 2, offset 58
Sent to partition 2, offset 59
Sent to partition 2, offset 60
Sent to partition 0, offset 25
Sent to partition 2, offset 61
Sent to partition 0, offset 26
Sent to partition 2, offset 62
Sent to partition 0, offset 27
Sent to partition 2, offset 63
Sent to partition 0, offset 28
Sent to partition 2, offset 64
Sent to partition 2, offset 65
Sent to partition 2, offset 66
Sent to partition 2, offset 67
Sent to partition 2, offset 68
Sent to partition 2, offset 69
Sent to partition 2, offset 70
Producer closed.
(bigdata) PS D:\EFREI\Data_Engineering\LAB3>
```

Simulated live data is now flowing into the pipeline, triggering transformations and disk writes in the Spark application.

7c Inspect the Parquet Output

Uses an interactive Spark shell to read the generated Parquet files and verify the pipeline's output.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.master("local[*]").getOrCreate()

df = spark.read.parquet("./spark-etl/output")
df.printSchema()
df.orderBy("window.start", "sensor").show(truncate=False)
```

`spark.read.parquet()` reads all files in the directory. Because Parquet is columnar, reading specific columns is highly efficient.

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB3> python
Python 3.9.25 (main, Nov 3 2025, 22:44:01) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from pyspark.sql import SparkSession
>>> spark = SparkSession.builder.master("local[*]").getOrCreate()
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/05/13 17:08:53 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.
>>> df = spark.read.parquet("./spark-etl/output")
>>> df.printSchema()
root
 |-- window: struct (nullable = false)
 |   |-- start: timestamp (nullable = true)
 |   |-- end: timestamp (nullable = true)
 |-- sensor: string (nullable = true)
 |-- avg_value: double (nullable = true)

>>> df.orderBy("window.start", "sensor").show(truncate=False)
```

window	sensor	avg_value
{2026-05-13 12:10:00, 2026-05-13 12:15:00}	humidity	24.465000000000003
{2026-05-13 12:10:00, 2026-05-13 12:15:00}	pressure	26.15421052631579
{2026-05-13 12:10:00, 2026-05-13 12:15:00}	temperature	24.541578947368418

Validates that the full ETL pipeline successfully ingested, transformed, windowed, and persisted the data accurately.

Reflection Questions

1. The `value` column from Kafka is in binary format. What happens if you call `.filter(col("value") > 30)` directly on the raw data stream before parsing it?

You will encounter calculation errors or unexpected behavior because the `value` column contains raw binary bytes, not numeric values. Spark cannot perform mathematical operations directly on binary strings and integers/double-precision numbers; you must first cast them to strings and parse them using functions like `from_json`.

2. You set `.trigger(processingTime="10 seconds")`. What trade-offs does this create between latency, throughput, and the number of generated Parquet files?

A 10-second processing time means micro-batches execute quickly, providing low latency for downstream consumers. However, compared to larger batches, this reduces overall throughput; moreover, since Spark creates a new file for each batch in every partition, this frequent triggering generates a large number of very small Parquet files. Longer intervals increase latency but improve throughput and result in fewer, larger files.

3. Explain in your own words why watermarks are necessary for windowed aggregation. What would happen without watermarks?

Watermarks define the maximum threshold for late data. In an unbounded data stream, without watermarks, Spark would never be able to determine when it is safe to close a time window. Consequently, it would have to permanently store the aggregate state of all past windows in memory, eventually leading to out-of-memory errors. Watermarks allow Spark to safely expire old states.

4. Your Spark job crashes after writing the 5th batch of data. When you restart the task, from which batch will Spark begin processing, and why?

It will start processing from the 6th batch. This is because the pipeline uses a checkpoint directory (`checkpointLocation`), which stores the stream processing state, including the exact Kafka offsets read so far. Upon restart, Spark reads the checkpoint to determine where processing was last interrupted, thereby avoiding the need to reprocess data starting from the very first offset.

5. You want to add a new transformation operation to the pipeline (e.g., converting Celsius to Fahrenheit). Where in the code should this operation be inserted? Is it necessary to change the checkpoint directory?

It should be inserted after the JSON parsing step or the filtering step, but must be placed *before* the windowed aggregation to ensure that the average is calculated based on the correct metrics. Adding a new transformation effectively changes the processing DAG and may affect the data schema. If this change involves stateful operations, Spark will be unable to reuse existing checkpoints and will require a new checkpoint directory (or clearing the old directory) to successfully restart the job.